AI Bootcamp — Week 2: Software Development Workflow

Monday — Student Exercise Guide

© 2026 Improving

OBJECTIVES

By end of Week 2, the candidate can:

- Apply the EDD discipline to a domain with right/wrong answers: software development artifacts
- Build out the DLP student-workspace skeleton skills through EC-first iteration, adding Context and Constraints that are demonstrably load-bearing
- Explain the pipeline's YAML file contracts: what each skill reads and writes, the fail-fast behavior on missing inputs, and the review write-back loop
- Direct the AI to write a Python evaluation script against their EC (deterministic checks first, LLM judge second) and run baseline + iteration cycles with it
- Trace quality degradation through a multi-stage pipeline and isolate the source
- Produce an evaluation-measured iteration history showing net improvement across at least one full skill

© 2026 Improving

TIMING

- **Monday–Friday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each day)

© 2026 Improving

MONDAY (9:00 AM – 5:00 PM): PIPELINE ORIENTATION AND BASELINE RUN

9:00 – 10:30 AM: THE SDLC PIPELINE AND ITS FILE CONTRACTS

Walk through the full workflow diagram. For each skill: what is the input, what is the output, and who (in a real engagement) is responsible for the quality of each step?

The YAML contracts are new this week and matter. The pipeline is not prompts passing prose to each other. Each skill writes a structured file the next skill reads:

- `create-epics-stories` writes `stories.yaml`
- `plan-story` reads `stories.yaml` and **fails immediately with a clear error if it is missing** (no stories, no plans), then writes `plan.yaml`
- `implement-story` reads `plan.yaml` (same fail-fast contract) and produces code
- `review-implementation` reads `stories.yaml` + `plan.yaml` + the code, writes `review.yaml`, and **updates `stories.yaml`** with `review_status` and `review_notes`

That last one is the literal feedback loop: stories flow forward through the pipeline, review flows back to update the original artifact.

Key discussion: the skeleton skills are not broken; they will produce output. The question is: what assumptions does the model make when Context and Constraints are absent? Those assumptions are the failure modes you will spend this week eliminating. The demo outputs in `demo-outputs/` show exactly what RT-only stubs produce: fluent, plausible, generic.

10:30 AM – 12:00 PM: FIRST PIPELINE RUN: OBSERVE, DON'T FIX

Clone the student workspace. Run all seven skills in sequence against a provided product description. Do not change any skill yet. Just run and observe.

For each output, note:

- What assumptions did the model make that were not in the skill?
- What did the model do that you did not ask it to do?
- What did the model fail to do that a human expert would have done automatically?

Record these observations. They are the raw material for your Evaluation Criteria.

12:00 – 1:00 PM: LUNCH

1:00 – 2:30 PM: THE THREE-WAY QUALITY BAR AND WRITING EC

Before writing any EC, run the contrast exercise. Read the three story sets in `sample-stories/` side by side:

© 2026 Improving

- **Good** (`pantry-inventory.md`): what specific, testable AC look like
- **Vague** (`vague-pantry-stories.md`): plausible-sounding, untestable, no edge cases
- **AI-generated without care** (`ai-generated-pantry-stories.md`): polished, confident, and wrong in a specific way: AC that prescribe implementation (PostgreSQL, JWT, Redux), epics too large, edge cases missing

Discussion: what makes the third set dangerous is that it *looks* better than the first. Fluency is not quality. This is why EC are written before building and why they must be measurable.

Now write EC for `create-prd` and `create-architecture`, by hand, before touching any skill. Use the failure observations from the morning run as input; those observed failures should appear as explicit failure criteria.

For `create-prd` : Define at least 4 EC, including at minimum:

- Required sections (what must be present in every PRD regardless of input)
- A criterion about specificity (the PRD should not use vague language like "easy to use" or "fast" without measurable definitions)
- A criterion about scope boundaries (what the PRD must explicitly exclude as out-of-scope)

For `create-architecture` : Define at least 4 EC. Consider what a solutions architect would always include that the model might omit without being told.

Reminder on vocabulary: these are Evaluation Criteria (EC), the quality bar for the skill's output. Acceptance Criteria (AC) are the product artifact that `create-epics-stories` produces. Never "success criteria."

2:30 – 5:00 PM: BASELINE EVALUATION FOR CREATE-PRD

Direct the AI to write a Python evaluation script for `create-prd` against your EC. You write the EC; the AI writes the code. Read the script before running it.

Deterministic first, LLM judge second:

- Deterministic checks (no model call: schema validation, section presence, field counts, format patterns) cover every EC that can be expressed as code
- LLM judge checks (send the skill output + the EC to a model, get a structured JSON verdict with per-criterion pass/fail and reasoning) only for EC that genuinely require judgment

If you find yourself reaching for an LLM judge for something that could be a regex, the EC needs refinement.

Run the baseline against the RT-only skeleton. Record scores. This is your Red state for Day 2. Commit the script to `evaluations/` .

© 2026 Improving